GoAuth Shield

Authentication API in Go: registration/login, bcrypt hashing, JWT access + rotating refresh tokens, protected routes, PostgreSQL, per-IP rate limiting.

Project layout

```
cmd/server/main.go            entrypoint, routing,
graceful shutdown
internal/config               env-based config +
validation
internal/database             pgx pool + embedded SQL
migrations
internal/models               domain types
internal/repository           PostgreSQL data access
(users, refresh tokens)
internal/auth                 bcrypt helpers, JWT
manager (+ tests)
internal/httpx                JSON response/decoding
helpers
internal/handlers             HTTP handlers (auth,
user)
internal/middleware           auth, rate limit,
logging, recovery, security headers
```

Quick start (Docker)

```
cp .env.example .env
make secrets            # paste both values into .env
(JWT_ACCESS_SECRET / JWT_REFRESH_SECRET)
go mod tidy             # generates go.sum (needs Go
```

```
1.22+)
docker compose up --build
```

Quick start (local Go)

```
cp .env.example .env    # set JWT secrets
docker compose up -d db # or use your own Postgres and
set DATABASE_URL
go mod tidy
make test
make run
```

Migrations run automatically at startup. Before publishing, replace the module path `github.com/yourname/goauth-shield` (go.mod and all imports) with your own.

Endpoints

Method	Path	Auth	Notes
POST	`/api/v1/auth/register`	–	`{email, password, name?}` → user + tokens
POST	`/api/v1/auth/login`	–	`{email, password}` → user + tokens
POST	`/api/v1/auth/refresh`	–	`{refresh_token}` → new token pair (rotation)
POST	`/api/v1/auth/logout`	–	`{refresh_token}` → 204

Method	Path	Auth	Notes
POST	`/api/v1/auth/logout-all`	Bearer	revokes all sessions
GET	`/api/v1/users/me`	Bearer	current user
GET	`/healthz`, `/readyz`	–	liveness / DB readiness

```
curl -s localhost:8080/api/v1/auth/register -H
'Content-Type: application/json' \
  -d '{"email":"ada@example.com","password":"correct-
horse-battery","name":"Ada"}'

curl -s localhost:8080/api/v1/auth/login -H 'Content-
Type: application/json' \
  -d '{"email":"ada@example.com","password":"correct-
horse-battery"}'

curl -s localhost:8080/api/v1/users/me -H
"Authorization: Bearer $ACCESS_TOKEN"

curl -s localhost:8080/api/v1/auth/refresh -H
'Content-Type: application/json' \
  -d "{\"refresh_token\":\"$REFRESH_TOKEN\"}"
```

Security design

- **Passwords:** bcrypt (cost 12 default), 8–72 byte limit (bcrypt truncates beyond 72), constant-ish timing on unknown emails, generic login errors.
- **JWT:** HS256 only (algorithm pinned), separate secrets and `typ` claim for access vs refresh, issuer and expiry enforced. Access tokens default to 15m.

- **Refresh rotation:** each refresh token has a `jti` stored in Postgres; using it revokes it atomically and issues a new pair. Re-presenting a used token revokes all of that user's sessions (theft detection).
- **Rate limiting:** global per-IP bucket plus a stricter bucket on register/login/refresh/logout. In-memory, so per instance.
- **Hardening:** request size limits, strict JSON, server timeouts, panic recovery, security headers, distroless non-root image.

Production notes

- Terminate TLS at a reverse proxy/load balancer and add HSTS there. Set `TRUST_PROXY=true` only if the proxy overwrites `X-Forwarded-For`; otherwise clients can spoof IPs.
- Use `sslmode=require` (or stricter) in `DATABASE_URL`, and keep secrets in a secret manager, not in git.
- Rotating JWT secrets invalidates all tokens. For zero-downtime rotation, add a `kid` header and support multiple keys.
- For a global rate limit across instances, use Redis or an edge limiter. Consider per-account login lockout and email verification/password reset as next steps.
- Clients should keep the refresh token in secure storage (Keychain/Keystore, or an HttpOnly cookie for browsers) and the access token in memory.